

- (a) Truthfully write the phrase *“I have read and understand the policies on the course website.”*

Solution: I have read and understand the policies on the course website. ■

- (b) Describe and analyze an $O(n \log n)$ time algorithm to compute (output) $\text{Pareto}(P)$.

Solution:

1. First, sort the points in P from highest to lowest by their x -values.
2. We then consider only the first point in the sorted list. Since the list is sorted, this point is guaranteed to be the point with the highest x -value. Therefore, there must not be any points to the right of it. We will always add this point to the Pareto set.
3. We then iterate through the remaining points in P . Because the points are sorted by x -value, we will always have considered all points to the right of the current point.
 - i. If the y -value of the current point is greater than the y -value of the previous point, we must add the current point to the Pareto set. Because we are traversing the points right to left and only adding points with greater y -values, we can guarantee that there will never be a point in the Pareto set with a point to the right of it that has a higher y -value.
 - ii. Before adding the point, we check if it's x -value is equal to the x -value of the previous point. If so, we remove the previous point from the Pareto set. This is necessary if we have two points with the same x -value, but the second point has a higher y -value.

```

Pareto( $P[p_1, \dots, p_n]$ ):
  Sort  $P$  by  $x$ -value, high to low
   $x_{prev}, y_{prev} \leftarrow (\infty, -\infty)$ 
   $P_{out} \leftarrow \emptyset$ 

  for  $i \leftarrow 1$  to  $n$ 
     $x_i, y_i \leftarrow p_i$ 
    if  $y_i > y_{prev}$ 
      if  $x_i = x_{prev}$ 
         $P_{out} \leftarrow P_{out} - (x_{prev}, y_{prev})$ 
       $P_{out} \leftarrow P_{out} \cup \{p_i\}$ 
       $x_{prev}, y_{prev} \leftarrow p_i$ 
  return  $P_{out}$ 

```

Sorting takes $O(n \log n)$ time. The for loop takes $O(n)$ time. Therefore, the algorithm has a time complexity of $O(n \log n)$. ■

- (c) Let $h = |\text{Pareto}(P)|$. Describe and analyze an $O(nh)$ time algorithm to compute $\text{Pareto}(P)$.

- Solution:**
1. First, find the point with the highest y -value in P . This will be the first point we add to the Pareto set.
 2. Iterate over the points in P and find the point with the highest y -value whose x -value is greater than the x -value of the previous point in the Pareto set. Add this point to the Pareto set.
 3. Repeat step 2 until there are no points to the right of the previous point in the Pareto set.

```

Pareto( $P[p_1, \dots, p_n]$ ):
   $x_{prev} \leftarrow -\infty$ 
   $P_{out} \leftarrow \emptyset$ 

  while true
     $p_{next} \leftarrow (null, -\infty)$ 
    for  $i \leftarrow 1$  to  $n$ 
       $x_i, y_i \leftarrow p_i$ 
      if  $x_i > x_{prev}$  and  $y_i > y_{p_{next}}$ 
         $p_{next} \leftarrow p_i$ 
     $x_{prev} \leftarrow x_{p_{next}}$ 
    if  $p_{next} = (null, -\infty)$ 
      break
     $P_{out} \leftarrow P_{out} \cup \{p_{next}\}$ 
  return  $P_{out}$ 

```

The while loop takes $O(h)$ time. The for loop takes $O(n)$ time. Therefore, the algorithm has a time complexity of $O(nh)$. ■

- (d) Describe and analyze an $O(n \log h)$ time algorithm to compute Pareto(P). For simplicity, you may assume that the value h is known in advance.

- Solution:**
1. Partition the points in P into k groups where $k = n/h$.
 2. Using the algorithm from part (b), compute the Pareto set for each group.
 3. Now we iterate through every point and select the point with the highest x -value to be our starting point. Then we iterate over each Pareto set and pick out the point that, relative to our previous point, has the closest lower x -value and a higher y -value. If we encounter any points with equal x -value, we select the one with a higher y -value. We add this point to the final Pareto set. Then we repeat this process until there are no points to the left of the previous point in the final Pareto set.

```

Pareto( $P[p_1, \dots, p_n], h$ ):
   $k \leftarrow \frac{n}{h}$ 
   $P_1, \dots, P_k \leftarrow \text{Partition}(P, k)$ 

  for  $i \leftarrow 1$  to  $k$ 
     $P_i \leftarrow \text{Pareto}_b(P_i)$ 

   $p_{prev} \leftarrow (-\infty, -\infty)$ 
  while true
     $p_{next} \leftarrow (-\infty, -\infty)$ 
    for  $i \leftarrow 1$  to  $k$ 
       $x_i, y_i \leftarrow$  modified b-search of  $P_i$  that returns the closest point less than  $x_{p_{prev}}$  and greater than  $y_{p_{prev}}$ 
      if  $x_i > x_{p_{next}}$ 
         $p_{next} \leftarrow P_i$ 
     $p_{prev} \leftarrow p_{next}$ 
    if  $p_{next} = (-\infty, -\infty)$ 
      break
     $P_{out} \leftarrow P_{out} \cup \{p_{next}\}$ 
  return  $\text{Pareto}_c(P_{out})$ 

```

The partitioning step takes $O(n)$ time. The first for loop takes $O(n/h)$ time and Pareto_b takes $O(h \log h)$ time. The while loop takes $O(h)$ time and the inner for loop takes $O(\frac{n}{h})$ time. The b-search takes $O(\log h)$ time. The whole thing takes $O(n + \frac{n}{h} \cdot h \log h + h \cdot \frac{n}{h} \cdot \log h) = O(n \log h)$ time. ■

Describe and analyze an $O(n \log n)$ time algorithm to determine whether or not any pair of circles intersect.

Solution: 1. **Sweep-line Status.** The sweep-line status is an ordered dictionary (b-tree) that contains the circle segments currently intersecting the sweep line, ordered by y -value. Circle segments will be defined clearly in 3.a.i

2. (a) **Event Points.** There are three types of event points:
- The leftmost edge of each circle. These mark the beginning of a circle when they are encountered by the sweep line.
 - The rightmost edge of each circle. These mark the end of a circle when they are encountered by the sweep line.
 - Any intersection points between two circle segments. These mark the points where the y -axis order of two circle segments swap.
- (b) **Event Queue.** The event queue is a priority queue (binary-heap) that contains the event points sorted by their x -value, low to high.

3. Operations

(a) Sweep-line Status

- Insertion.** When the sweep-line encounters the left edge of a circle, we insert two circle segments into the sweep-line status. These are initialized with the y -value of the edge of the circle. We will mark one as the above circle segment and it will follow the top edge of the circle while the other will be marked as below and follow the bottom edge. After insertion, we check for any intersections between the new circle segments and any adjacent circle segments in the sweep-line status.
- Deletion.** When the sweep-line encounters the right edge of a circle, we remove both circle segments associated with that circle from the sweep-line status. After deletion, we check for any intersections between circle segments that are newly adjacent.
- Swap.** When the sweep-line encounters an intersection point, we swap the order of the intersecting circle segments in the sweep-line status. Then we update each of the swapped segments with the y -value of the intersection. After the swap, we check for any intersections between the circle segments that are newly adjacent.

(b) Event Queue

- Initialization.** We initialize the event queue with the event points of the left and right edges of each circle. We can calculate these points with $x_{left} = x_i - r_i$ and $x_{right} = x_i + r_i$, where x_i is the x -value of the center of the circle and r_i is the radius.
- Processing.** We process the event queue by popping the next event point and updating the sweep-line status accordingly. If it's a left edge, insert the circle segments. If it's a right edge, delete the circle segments. If it's an intersection point, swap the circle segments.

- iii. **Insertion.** When a Sweep-line Status operation finds intersection points between circle segments, we insert a new event point into the event queue for each one. The event point will remember the two circle segments that are intersecting, the x and y values of the intersection, and will be prioritized by the x -value of the intersection point.

4. Time Complexity.

- (a) **Event Points.** First we'll calculate the number of event points. Each circle is guaranteed to have an event for each of its edges, left and right, so $2n = O(n)$ events. Each circle can only intersect with another circle a maximum of two times. Therefore, the number of events for intersections can be no more than $\sum_{i=1}^n 2(n-i) = O(n)$. Therefore, events take a total of $O(n)$ time.
- (b) **Data Structures.**
 - i. **Sweep-line Status.** The insertion and deletion operations of the sweep-line status will take $O(\log n)$ time. The swap operations will take $O(1)$ time.
 - ii. **Event Queue.** The insertion operations of the event queue will take $O(\log n)$ time. Popping will take $O(1)$ time.
- (c) **Total.** The total time complexity of the algorithm will be the number of events times the running time of each event. $O(n) * O(\log n) = O(n \log n)$.



Describe an $O(n \log n)$ time algorithm that, given the polygon P and the value $w_{\min}$, determines whether the river will flood.

Solution: 1. **Proof.** In order to assert that it suffices to check only the vertical cuts $x = x_0$ for which x_0 is the x-coordinate of a polygon vertex, I must prove that $w_{\min}$ will always be located at the x-coordinate of a polygon vertex. To do this, I will consider w_i at every x-coordinate (x_i) that isn't a polygon vertex and show that it will always be greater than or equal to $w_{\min}$. By the definition of the problem, we know that x_i will always be bounded by two vertices. We'll label the x-coordinates and widths of these vertices as x_1, x_2, w_1 , and w_2 where $x_1 < x_2$. We begin by considering the rate of change of w (Δ_w) between any two vertices. There are three cases:

$\Delta_w > 0$ In this case, w is increasing between the two vertices. Therefore, it must be the case that $w_1 < w_i < w_2$ for any x_i between the two vertices. Clearly w_i must not be $w_{\min}$ because $w_1 < w_i$.

$\Delta_w = 0$ In this case, w is constant between the two vertices. Therefore, it must be the case that $w_1 = w_i = w_2$ for any x_i between the two vertices. While it might be the case that $w_i = w_{\min}$, it suffices to check w_1 or w_2 because they are all equal.

$\Delta_w < 0$ In this case, w is decreasing between the two vertices. Therefore, it must be the case that $w_1 > w_i > w_2$ for any x_i between the two vertices. Clearly w_i must not be $w_{\min}$ because $w_2 < w_i$.

For any x_i that isn't a polygon vertex, it will always be the case that w_1 or w_2 is less than or equal to w_i . Therefore, $w_{\min}$ will always be captured at a vertex and it suffices to check only the vertical cuts $x = x_0$ for which x_0 is the x-coordinate of a polygon vertex.

2. Algorithm.

(a) **Stored Values.** As we go, we must store the current width of the river w_i as well as the minimum width so far $w_{\min}$. We will initialize both values to be the distance between the first vertices in $x-$ and $x+$. We will also need to store the current rate of change of the width Δ_w . We will initialize this value to 0. The final value we'll store is the previous x-coordinate (x_{prev}). We will initialize this value to the x-value of the first vertex in $x-$.

(b) **Event Queue.** Let $I = \{I_1, \dots, I_k\}$ (where k is the number of islands) denote the set of polygons that make up the islands in the river. For every vertex in I and P , we find the slope Δ_r and Δ_l of the lines on each side and store them as an event point. For any endpoints we can simply store 0. For start and endpoints of I_i , we must store two slopes exiting and entering these vertices respectively. We initialize the event queue to be a priority queue containing the event points for each vertex in each polygon. Each event point will be prioritized by the x-value of the vertex. Every time we pop an event point, we will do the following:

Update the current width of the river $w+ = \Delta_w * (x - x_{prev})$. If the new width is less than $w_{\min}$, we set $w_{\min} = w$. We then update the previous x-coordinate $x_{prev} = x$.

Finally, we update the rate of change of the width based on which polygon the event point's vertex came from.

$$x+) \quad \Delta_w+ = \Delta_r - \Delta_l$$

$$x-) \quad \Delta_w- = \Delta_r - \Delta_l$$

I_i) If the event point is the start of I_i , $\Delta_w+ = \Delta_{rb} - \Delta_{rt}$ where Δ_{rt} is the line on the top of the polygon and Δ_{rb} is the line on the bottom. If the event point is the end of I_i , $\Delta_w- = \Delta_{lb} - \Delta_{lt}$. If the event point is on the top of I_i , $\Delta_w- = \Delta_r - \Delta_l$. If the event point is on the bottom of I_i , $\Delta_w+ = \Delta_r - \Delta_l$.

After processing all event points, we check if our $w_{min} < w_{min}$. If so, then we know the river will flood.

■